**CHARLES DARWIN UNIVERSITY**

**Assessment 1 - Group Project Proposal and Design Plan**

**Group Number – SYD7**


| Student Name | Student ID |
|---|---|
| Guvvala Thejeswar Reddy | 384042 |
| Julakanti Thirumala Raja Sumanth Reddy | 386402 |
| Dhruv Pramod Vakharia | 385234 |

Assessment 1: Project Proposal and Design (10%)

| Field | Detail |
|---|---|
| Project Title | Flight Delay Prediction: A Predictive Analytics and Forecasting Approach |
| Project Theme | Theme 2 – Predictive Analytics and Forecasting |
| Unit Code | PRT661 |
| Lecturer | Reem Sherif |
| Domain | Aviation Operations & Transport Analytics |
| Primary Dataset | U.S. DOT BTS Airline On-Time Performance Dataset (2018–2023) |
| Tools | Python, DuckDB, MinIO, scikit-learn, XGBoost, LightGBM, Plotly Dash, Apache Airflow |

## Table of Contents

## 1. Project Overview and Objectives

There is a significant economic cost associated with flight delays. The societal benefits of a 10% decrease in U.S. flight delays would amount to USD 17.6 billion, and those benefits would be USD 38.5 billion with a reduction of 30%, after taking into consideration airline costs, passenger time, and lost productivity, find Peterson et al. (2013). This project involves the development of a batch predictive analytics system that classifies whether a scheduled U.S. domestic flight is expecting a delay and estimates the duration of the expected delay based on historical flight data that is enhanced with weather, carrier and airport context data, processed in a five-step pipeline and presented to airline operations teams on an interactive dashboard.

Lambelho et al. (2020) present evidence of machine-learning models outperforming simple scheduling heuristics for predicting delay/cancellation with carrier behaviour and contextual features; and that gradient-boosting ensembles (XGBoost, LightGBM, CatBoost) with weather features can achieve strong accuracy on large-scale aviation datasets, as demonstrated in Dai (2024). This project brings these results into practice in the form of a reproducible, open-source pipeline.

Objectives: acquire, clean and merge BTS flight records, NOAA weather and airport metadata (2018–2023); perform EDA on BTS flight delay patterns across carriers, airports, routes, seasons, and NOAA weather; prepare temporal and contextual predictive features; train and test classifiers (Logistic Regression, Random Forest, XGBoost, LightGBM) for binary prediction delay or not; create a regression sub-model forecasting duration of delay in minutes; provide the results through an operational dashboard; and apply ethical, privacy-aware, and reproducible practice throughout.

This is a Data Science endeavour, not just a Data wrangling/ML exercise, everything from acquiring and storing data, to wrangling, exploring, modelling, to decision-oriented visualization. The decision that is supported is well-specified and measurable: will this flight depart on time, and if not, about how many minutes late will it be?

## 2. Theme Justification

This project falls into Theme 2 – Predictive Analytics and Forecasting: batch, structured, tabular data for supervised learning. The following six, mandated but often overlooked, components of a pipeline acquisition, storage, processing, analytics/ML, visualisation and workflow automation are all addressed as well as data-quality assessment, data Five Vs analysis, and dashboard design for stakeholders.

## 3. Problem Statement

Real world operations rely on rules-of-thumb based aviation delay prediction that assumes a stochastic weather process, fails to take network interactions between flights into account and is based on an aviation sector information system with considerable heterogeneity across carriers and airports. The research problem is as follows: Can a machine-learning system predict, before departure, whether a flight will be delayed and, if it is, when estimate the duration of the delay based on historical flight performance information that includes the weather at the time of flight, the carrier information, and information about the airports where the flight takes off and lands?

The flight delay patterns in the COVID-19 period are materially different from those of the previous years, Faiza and Khalil (2023) demonstrate, leading to temporal validation: this project employs walk-forward cross validation and also does not consider the data for the COVID period (2020–2021) with other normal years. The authors of Li et al. (2024) conclude that the topology of airports, namely the connectivity of a hub and its node centrality are measurable influences for the propagation of delays and thus their graph-derived features, such as node centrality and hub classification, are motivated for use in the feature set.

### 3.1 Target Users and Business Case

While the primary users will be the airline operations control centres and the airport slot coordinators, the secondary users will be the operations, passenger facing apps and the airline revenue management. Even for small delays of a few minutes on a system-wide level, the benefits in recuperated welfare are measured in billions, while on a carrier level

the predictive delay intelligence makes pro-active crew reallocation and gate reallocation, and passenger rebooking, possible.

## 4. Initial Architecture Diagram

It is built around a batch analytics pipeline in five open-source layers (seen in Figure 1).

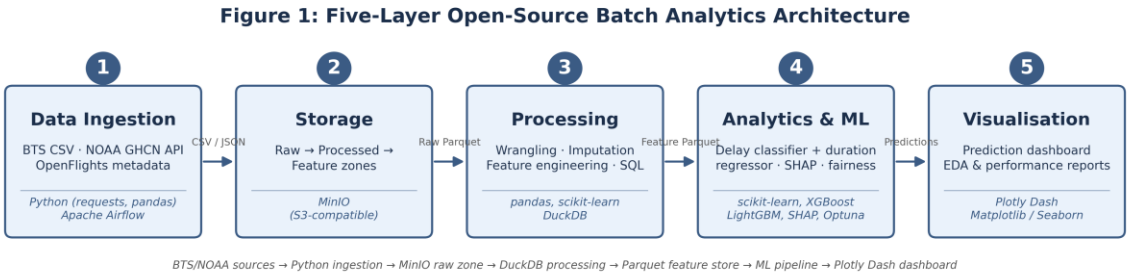


**Figure 1: Five-Layer Open-Source Batch Analytics Architecture**

The data flow is: BTS/NOAA sources to Python ingestion to MinIO raw zone to DuckDB processing to Parquet feature store to ML pipeline to Plotly Dash dashboard. The full history of all transformations is recorded in a data lineage document and uploaded onto GitHub, based on the work and suggestions of Stoudt et al. (2024) about an auditable and responsible data science workflow.

## 5. Workflow Plan

The project is broken down into four sprints that correspond to the assessment calendar, and planning, review, and retrospective ceremonies are conducted every two weeks and recorded in Microsoft Teams and tracked on Jira.

**Table 1: Sprint workflow aligned with PRT661 assessment calendar.**

| Sprint | Weeks | Key Activities | Deliverable |
|---|---|---|---|
| 1 | 1–3 | Dataset audit; repo/Jira setup; EDA kick-off; initial architecture | Assessment 1 Proposal |
| 2 | 4–7 | Cleaning pipeline; feature engineering; baseline models; Airflow DAG; MinIO config | Assessment 2 Progress Report |

| | | | |
|---|---|---|---|
| 3 | 8–11 | XGBoost/LightGBM/RF; Optuna HPO; SHAP; fairness check; SMOTE; dashboard prototype | Assessment 3 Presentation |
| 4 | 12–14 | Dashboard finalisation; automated reporting; ethics review; lineage docs; README | Assessment 4 Final Report |

## 6. Dataset Selection and Data Characteristics

### 6.1 Primary Dataset

The key data source is the U.S. DOT Bureau of Transportation Statistics (BTS) Airline On-Time Performance Dataset (transtats.bts.gov) that includes scheduled/actual times, delay minutes by cause, origin/destination, tail numbers, and distance for air transportation within the United States. The extract from 2018–2023 has about 36 million records in 18 carriers and 373 airports (~12 GB CSV; ~2.5 GB in Parquet format). Weather and geographic/hub additional data are provided by NOAA GHCN-Daily weather and the OpenFlights airport database.

**Primary Dataset:**

BTS Airline On-Time Performance Dataset

https://www.transtats.bts.gov

**Supplementary**

NOAA GHCN-Daily weather

https://www.ncei.noaa.gov/products/land-based-station/global-historical-climatology-network-daily

OpenFlights airport metadata

https://openflights.org/data.php

### 6.2 Five Vs Analysis

**Table 2: Five Vs analysis of the flight delay prediction dataset.**

| Dimension | Assessment and Architectural Implication |
|---|---|
| Volume | ~36M records / ~12 GB CSV to ~2.5 GB Parquet; DuckDB in-process SQL; MinIO three-zone storage. |
| Velocity | Batch: BTS monthly, NOAA daily via API. No streaming needed; Airflow manages scheduling. |
| Variety | Structured CSV, semi-structured JSON (NOAA), relational lookups (OpenFlights) unified via DuckDB. |
| Veracity | High — mandated federal reporting. ~1.3% missing delay-cause codes; 2020–2021 needs stratification. |
| Value | Supports delay classification, duration regression, carrier benchmarking, seasonal/airport monitoring. |

## 7. Task Allocation and Team Structure

This is because the team adopts a distributed ownership model: each member owns a layer of the pipeline, and is responsible for designing, implementing, documenting (using GitHub/Jira tickets, see Table 3), and testing this layer. Sprint ceremonies are conducted biweekly on Microsoft Teams and each portion of work is backed by individual commit, pulls request and or jira tasks for contribution evidence.

**Table 3: Team roles and deliverables.**

| Member | Role | Deliverables |
|---|---|---|
| 1 | Data Engineer / Pipeline Lead | BTS + NOAA ingestion; MinIO config; Airflow DAG; ETL; data quality audit |
| 2 | Data Analyst / EDA Lead | EDA notebooks; delay visualisations; DuckDB SQL analytics |
| 3 | ML Engineer | Model training; Optuna HPO; walk-forward CV; SHAP; fairness evaluation |
| 4 | Dashboard Developer | Plotly Dash app; prediction UI; performance views; scheduled reports |
| 5 | Project Manager / | Jira management; architecture diagrams; ethics |

| | | |
|---|---|---|
| | Ethics Lead | review; README/risk register |

## 8. Initial Risk Analysis

**Table 4: Risk register with likelihood, impact, and mitigation strategies.**

| Risk | Likelihood | Impact | Mitigation |
|---|---|---|---|
| Dataset size exceeds local memory | High | Medium | Parquet + DuckDB; prototype on 2022–2023 subset |
| COVID-19 period causes distribution shift | High | High | Temporal stratification; walk-forward CV |
| Class imbalance (~80/20) biases classifier | Certain | High | SMOTE/class weighting; evaluate via AUC-ROC/F1 |
| NOAA API rate limits or outages | Medium | Medium | Cache responses; fallback to NOAA bulk FTP files |
| Team member non-participation | Medium | High | GitHub commit evidence; redistribute tasks per sprint |
| Scope creep into real-time streaming | Medium | Medium | Strict backlog governance; Jira-documented scope changes |

## 9. Ethics, Privacy, and Security Considerations

### 9.1 Ethics and Fairness

The BTS data does not include any PII for passengers, but does provide data on the carriers and routes that could show disparities influenced by socioeconomic access to air travel. Drawing on Mehrabi et al. (2021) fairness framework, model performance will be broken down based on carrier type, airport hub classification, and geography of the route, and gaps reported transparently, with post hoc interventions, ranging from equalised-odds adjustment (as an example), as necessary.

### 9.2 Privacy

All the data sources are openly licensed and public: BTS data under the U.S. federal open data mandate, GHCN NOAA data as public domain and OpenFlights data under the

ODbL. No sensitive data requiring consent is gathered; only variables that are relevant in prediction are stored in the data lake, and none of the raw files with people's tail numbers is publicly published.

### 9.3 Security

All the following are in place: Shared repository access (GitHub branch-protection) via keys currently in the hands of all team members which are excluded in the .gitignore (API keys are stored as environment variables), MinIO role-based access control is used for access to the repositories, and all communication is by means of university-sanctioned Microsoft Teams/Outlook. The data lineage log is created and kept throughout the process, with every data transformation committed, following the approach of Stoudt et al. (2024).

## 10. References


Dai, M. (2024). A hybrid machine learning-based model for predicting flight delay through aviation big data. Scientific Reports, 14, Article 4603. https://doi.org/10.1038/s41598-024-55217-z

Faiza, & Khalil, K. (2023). Airline flight delays using artificial intelligence in COVID-19 with perspective analytics. Journal of Intelligent & Fuzzy Systems, 44(4), 6631–6653. https://doi.org/10.3233/JIFS-222827

Lambelho, M., Mitici, M., Pickup, S., & Marsden, A. (2020). Assessing strategic flight schedules at an airport using machine learning-based flight delay and cancellation predictions. Journal of Air Transport Management, 82, Article 101737. https://doi.org/10.1016/j.jairtraman.2019.101737

Li, Q., Wu, L., Guan, X., & Tian, Z. (2024). Interplay of network topologies in aviation delay propagation: A complex network and machine learning analysis. Physica A: Statistical Mechanics and its Applications, 638, Article 129622. https://doi.org/10.1016/j.physa.2024.129622

Mehrabi, N., Morstatter, F., Saxena, N., Lerman, K., & Galstyan, A. (2021). A survey on bias and fairness in machine learning. ACM Computing Surveys, 54(6), 1–35. https://doi.org/10.1145/3457607

Peterson, E. B., Neels, K., Barczi, N., & Graham, T. (2013). The economic cost of airline flight delay. Journal of Transport Economics and Policy, 47(1), 107–121. https://doi.org/10.3828/jtep.2013.47.1.107

Stoudt, S., Jernite, Y., Marshall, B., Marwick, B., Sharan, M., Whitaker, K., & Danchev, V. (2024). Ten simple rules for building and maintaining a responsible data science workflow. PLOS Computational Biology, 20(7), Article e1012232. https://doi.org/10.1371/journal.pcbi.1012232

## Appendices

## Appendix A: Data Dictionary

**Table 5: Key fields by source dataset**

| Dataset | Field | Description | Type |
|---|---|---|---|
| BTS | FlightDate | Scheduled date of the flight | Date |
| BTS | Reporting_Airline | IATA carrier code | Categorical |
| BTS | Origin / Dest | Origin and destination airport IATA codes | Categorical |
| BTS | CRSDepTime / CRSArrTime | Scheduled local departure / arrival time | Time |
| BTS | DepDelayMinutes | Departure delay in minutes (0 if early/on-time) | Numeric |
| BTS | ArrDelayMinutes | Arrival delay in minutes — regression target | Numeric |
| BTS | Cancelled / CancellationCode | Cancellation flag and reason | Boolean / Categorical |
| BTS | CarrierDelay, WeatherDelay, NASDelay, SecurityDelay, LateAircraftDelay | Delay minutes attributed to each cause | Numeric |
| BTS | Distance | Route distance in miles | Numeric |
| NOAA GHCN-Daily | STATION, DATE | Weather station identifier and observation date | Categorical / Date |
| NOAA GHCN-Daily | PRCP, SNOW, TMAX, TMIN, AWND | Precipitation, snowfall, temperature extremes, wind speed | Numeric |
| OpenFlights | airport_id, IATA, ICAO | Airport identifiers used | Categorical |

| | | | |
|---|---|---|---|
| | | to join to BTS Origin/Dest | |
| OpenFlights | latitude, longitude | Airport coordinates, used for nearest-station weather join | Numeric |

**Appendix B: Detailed Pipeline Process Design**

Instead of giving the code for an actual implementation of each of the 5 "architecture" layers outlined in section 4, each layer is detailed at the design level, that is, what the layer is responsible for and what the layer passes on to the next.

- Lineage tracking of downloaded files is performed with retention of checksums, and with scheduled Airflow tasks pulling monthly BTS extractions over HTTPS, NOAA GHCN-Daily records over bulk FTP with API fallback, and finally picking up of the static OpenFlights CSV.
- Storage (MinIO): three buckets to store the data: the raw-zone (where the data lands before being immutable), curated-zone (where the data is stored after DuckDB cleanup and typed), and feature-zone (model-ready, versioned by training run, feature tables).
- Processing (DuckDB): SQL transformations deduplicate data on flight-leg keys, normalise the local-to-UTC conversion for departure and arrival time, join the flight information to the nearest station for the date of the flight, join the flight information to the airport data using IATA code, and impute missing delay-cause codes according to a set of rules.
- Temporal features (hour of day, day of week, holiday flag), carrier/airport delay-rate features (rolling the last 7 days) and features built from the graph (hub-centrality features based on Li et. al. (2024)) are added as features; models are trained on walk-forward windows and models are registered with their feature-set version for reproducibility.

  Visualisation/automation: Visualisation is built into the Plotly Dash app, which is only fed by the feature-zone and a predictions table only updated by the nightly batch-scoring task scheduled by Airflow.

## Appendix C: Model Development and Evaluation Plan

**Table 6: Planned models and evaluation metrics.**

| Task | Candidate Models | Key Hyperparameters | Evaluation Metrics |
|---|---|---|---|
| Delay classification (delay / no delay) | Logistic Regression (baseline), Random Forest, XGBoost, LightGBM | Max depth, learning rate, n_estimators, class weight / scale_pos_weight | AUC-ROC, F1, precision/recall, Brier score, calibration curve |
| Delay-duration regression (minutes) | XGBoost Regressor, LightGBM Regressor | Max depth, learning rate, n_estimators, objective function | MAE, RMSE, $R^2$ |
| Fairness check (per Mehrabi et al., 2021) | Same trained models, sliced by group | Grouping variables: carrier type, hub classification, route geography | Equalised-odds gap, demographic parity gap, per-group AUC/F1 |

## Appendix D: Technology Justification Matrix

**Table 7: Tool choices and alternatives considered**

| Tool Selected | Alternative(s) Considered | Rationale |
|---|---|---|
| DuckDB | Apache Spark | In-process SQL engine is sufficient for the ~2.5 GB Parquet feature store; avoids cluster setup and operational overhead |
| MinIO | AWS S3 | Self-hosted and free during development; exposes an S3-compatible API, so migration to a cloud bucket later is low-risk |
| XGBoost / | CatBoost, neural | Strong reported performance on large tabular |

| | | |
|---|---|---|
| LightGBM | networks | aviation data (Dai, 2024; Lambelho et al., 2020) with faster training and SHAP-based interpretability |
| Plotly Dash | Streamlit, Tableau | Full callback control for interactive filtering by carrier/airport/date, and remains open-source and Python-native |
| Apache Airflow | Prefect, cron | Mature DAG scheduling with built-in retry semantics and run history, needed for the nightly batch-scoring workflow |